

# Activation Functions

≡ Week

Week 3

## Domain and Range:

The domain of a function is the set of values that the independent variable can take on, while the range is the set of values that the dependent variable can have.

| Function                       | Domain                           | Range                                |
|--------------------------------|----------------------------------|--------------------------------------|
| $y = \arcsin(x)$               | $-1 \leq x \leq 1$               | $-\pi/2 \leq y \leq \pi/2$           |
| $y = \arccos(x)$               | $-1 \leq x \leq 1$               | $0 \leq y \leq \pi$                  |
| $y = \arctan(x)$               | $-\infty < x < \infty$           | $-\pi/2 < y < \pi/2$                 |
| $y = \operatorname{arccsc}(x)$ | $x \leq -1 \text{ or } x \geq 1$ | $-\pi/2 \leq y \leq \pi/2, y \neq 0$ |
| $y = \operatorname{arcsec}(x)$ | $x \leq -1 \text{ or } x \geq 1$ | $0 \leq y \leq \pi, y \neq \pi/2$    |
| $y = \operatorname{arccot}(x)$ | $-\infty < x < \infty$           | $0 < y < \pi$                        |

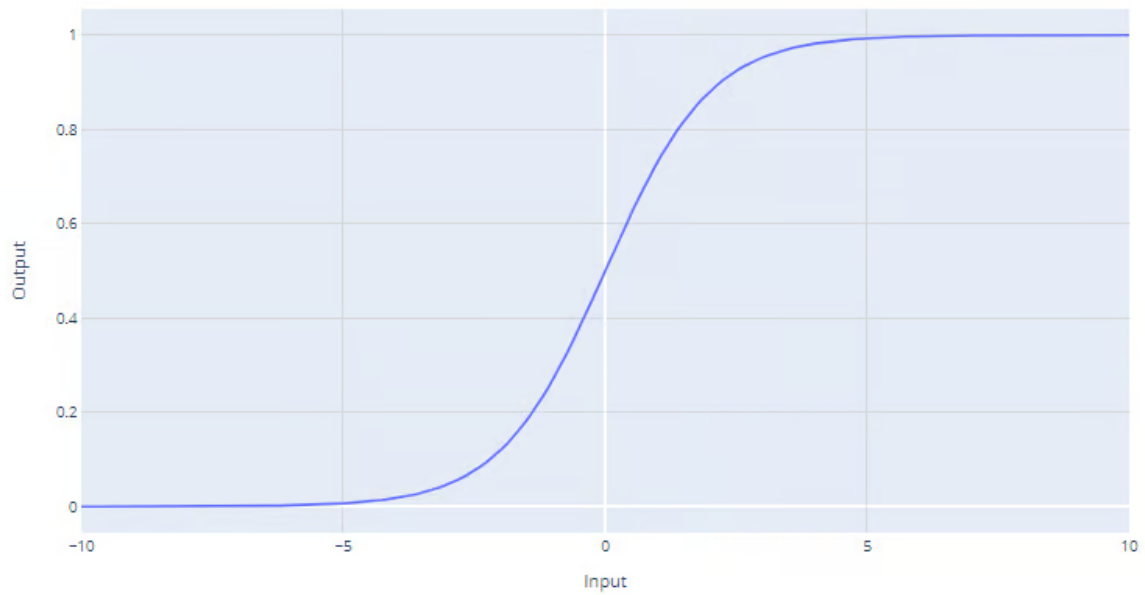
**Domain** → The set of all possible input values for a function.

**Range** → The set of all possible output values for a function.

Activation functions introduce non-linearities, allowing neural networks to learn highly complex mappings between inputs and outputs.

## Sigmoid activation (Also known as S-shaped function)

### Sigmoid Activation Function



*Sigmoid activation function.*

The sigmoid activation function, often represented as  $\sigma(x)$ , is a smooth, continuously differentiable function that is historically important in the development of neural networks. The sigmoid activation function has the mathematical form:

$$f(x) = 1 / (1 + e^{-x})$$

$$\text{Sigmoid} = \frac{1}{1 + e^{-x}}$$

★ Highest value of the derivation function of sigmoid function is 0.25.

The output of sigmoid function will always be between 0 and 1.

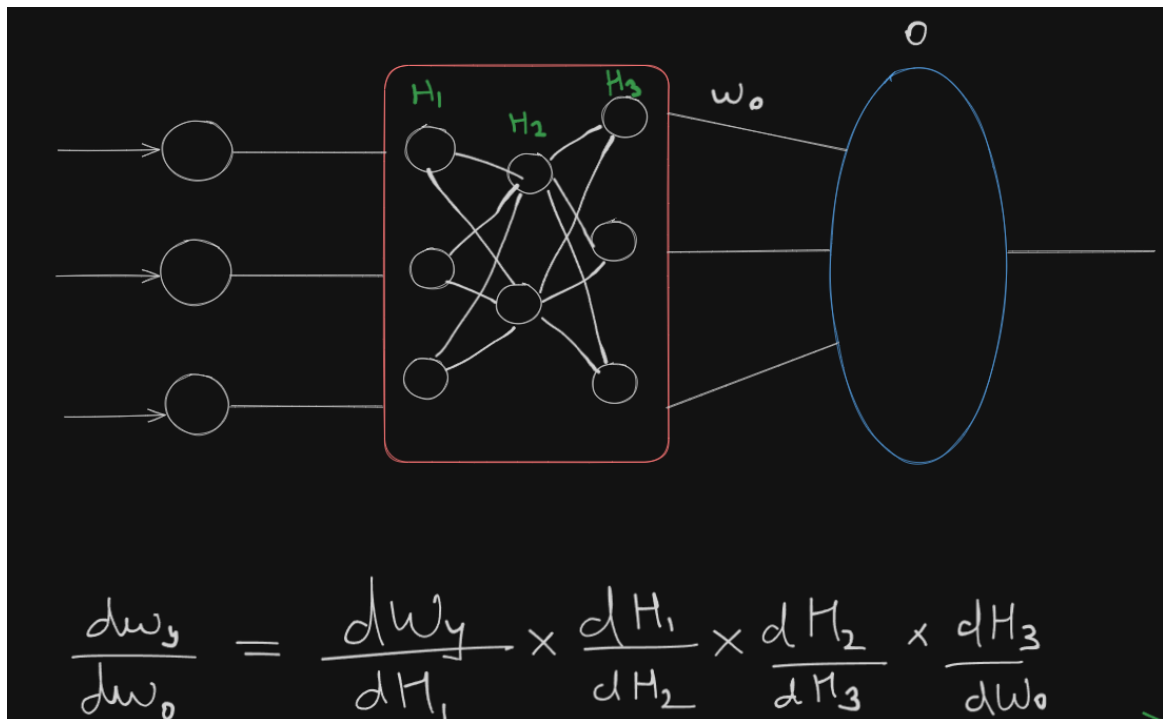
In case of binary classification , the activation function of the output neuron will always be sigmoid function. Because the output of sigmoid function will always be between 0 and 1.

If  $y$  is function of  $r$  and  $r$  is function of  $x$ , then:

$$\frac{dy}{dx} = \frac{dy}{dr} \times \frac{dr}{dx}$$

It is called as chain of differentiation.

However, sigmoid units suffer from the "vanishing gradient" problem that hampers learning in deep neural networks.



for multiple hidden layers, every hidden layer will contribute to a different slope term ( $dl/dw$ ) getting multiplied by the other slope terms.

**Vanishing Gradient** → As the input values become significantly positive or negative, the function saturates at 0 or 1, with an extremely flat slope. In these regions, the gradient is very close to zero. This results in very small changes in the weights during backpropagation, particularly for neurons in the earlier layers of deep networks, which makes learning painfully slow or even halts it. This is referred to as the vanishing gradient problem in neural networks.

$$\frac{dw_3}{dw_0} = \frac{dw_3}{dH_1} \times \frac{dH_1}{dH_2} \times \frac{dH_2}{dH_3} \times \frac{dH_3}{dw_0}$$

0.25    $\times$    0.25    $\times$    0.25    $\times$    0.25

$\rightarrow 0.004 = \frac{dl}{dw}$

$\eta \times \frac{dl}{dw} = 0.1 \times 0.004 = 0.0004$

let's say that learning rate = 0.1  
 so the Q value would be 0.0004  
 which is much less incremental value.  
 It signifies that the weight is not updating  
 that much. This is vanishing gradient.

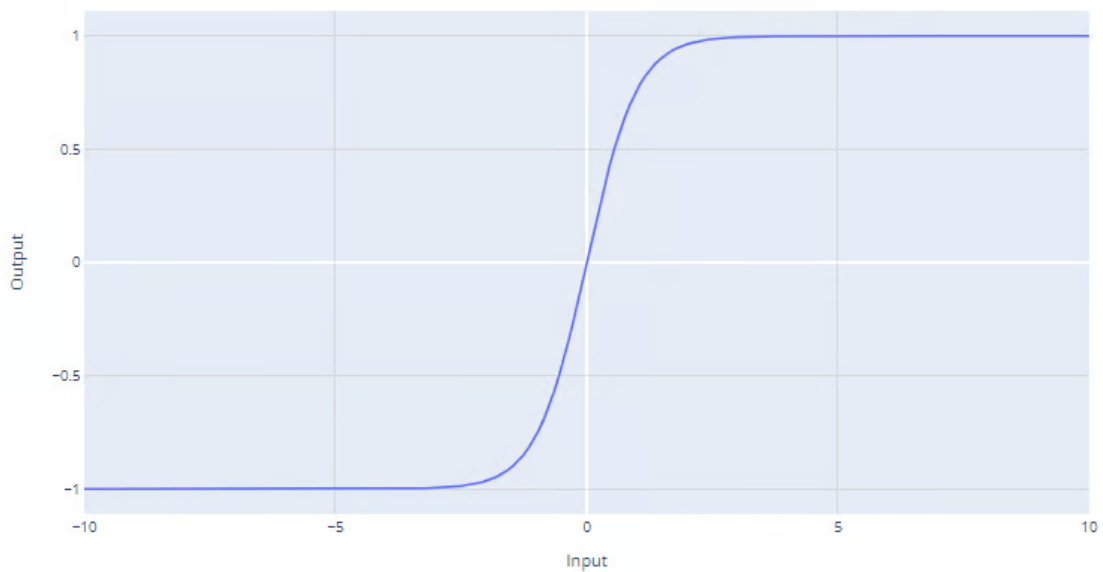
*Vanishing Gradient*

- Sigmoid is used for binary classification.
- It is used only on output layer.

1. The max value of the derivative of sigmoid function is 0.25
2. When we need to find the overall change in the output with respect to the change in the weights, we shall be using the chain rule of derivatives.
3. Even in the best case scenario, the value of 0.25 would be multiplied multiple times resulting in a very small number. This small number when multiplied by eta would result in even smaller number since the normal value of eta is 0.1
4. Because of this a very small number gets added or subtracted to the original value of weights because of which they practically do not change.
5. This unintentional stoppage in change of weights does not allow the model to be improved further and is called as problem of vanishing gradient.

## Tanh (hyperbolic tangent) activation function

### Hyperbolic Tangent Activation Function



*Tanh activation function.*

- The value of the derivative of the tanh function can go upto 1.
  - $\text{derivative}(\tanh) \leq 1$

The tanh (hyperbolic tangent) activation function is defined as:

$$f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$$

$$\tanh f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

The tanh function outputs values in the range of -1 to +1. This means that it can deal with negative values more effectively than the sigmoid function, which has a range of 0 to 1.

It can help the learning algorithm converge faster.

Tanh help to solve non zero centered problem of sigmoid function. Tanh squashes a real-valued number to the range  $[-1, 1]$ . It's non-linear too.

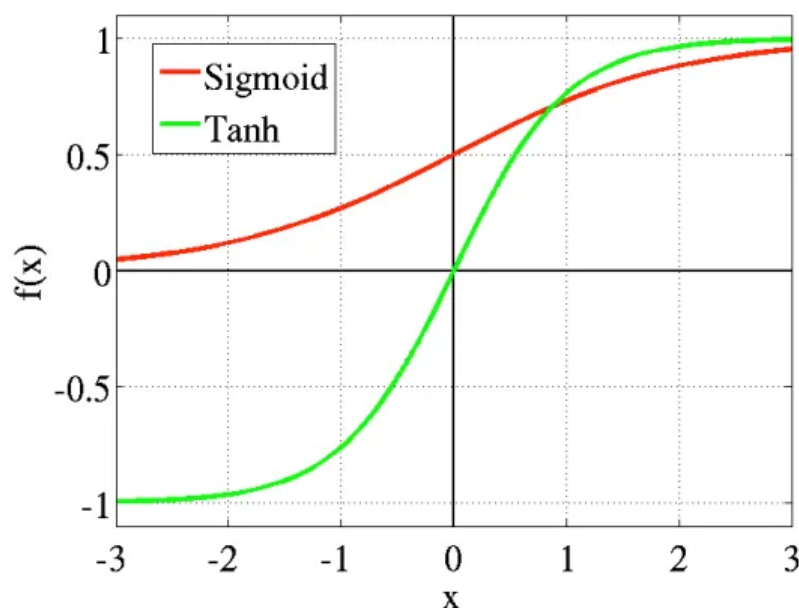
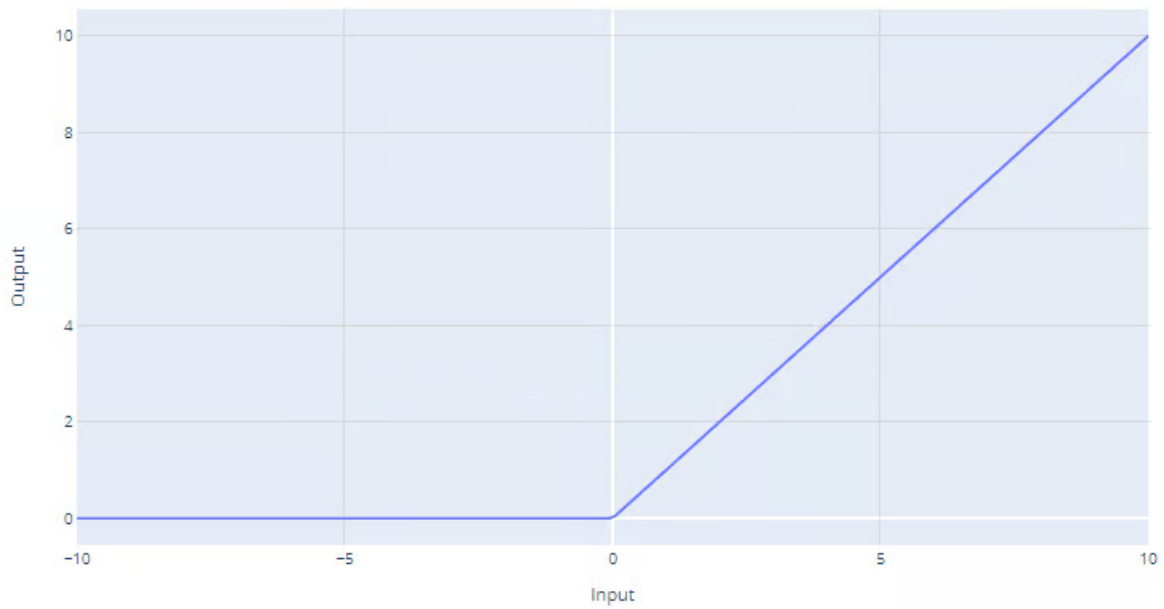


Fig: tanh v/s Logistic Sigmoid

Despite these advantages, the tanh function still suffers from the vanishing gradient problem. During backpropagation, the gradients of the tanh function can become very small (close to zero). This can drastically slow down the training process and can lead to poor convergence properties.

## ReLU (Rectified Linear Unit) activation

## ReLU Activation Function



*Rectified Linear Unit (ReLU) activation function.*

| Function                                                     | Derivative                                                 |
|--------------------------------------------------------------|------------------------------------------------------------|
| $R(z) = \begin{cases} z & z > 0 \\ 0 & z \leq 0 \end{cases}$ | $R'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$ |
|                                                              |                                                            |

The Rectified Linear Unit (ReLU) activation function has the form:

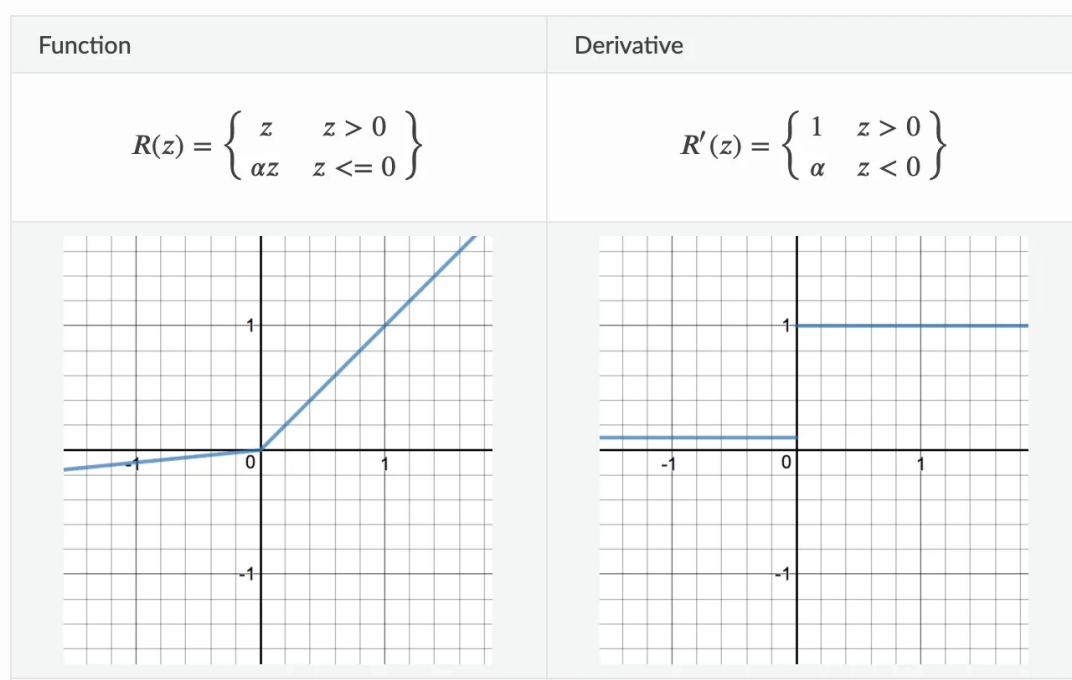
$$f(x) = \max(0, x)$$

It thresholds the input at zero, returning 0 for negative values and the input itself for positive values.

For inputs greater than 0, ReLU acts as a linear function with a gradient of 1. This means that it does not alter the scale of positive inputs and allows the gradient to pass through unchanged during backpropagation. This property is critical in mitigating the vanishing gradient problem.

Sometime some gradients can be fragile during training and can die. That leads to dead neurons. In another words, for activations in the region ( $x < 0$ ) of ReLU, gradient will be 0 because of which the weights will not get adjusted during descent. That means, those neurons which go into that state will stop responding to variations in error/ input ( simply because gradient is 0, nothing changes ).

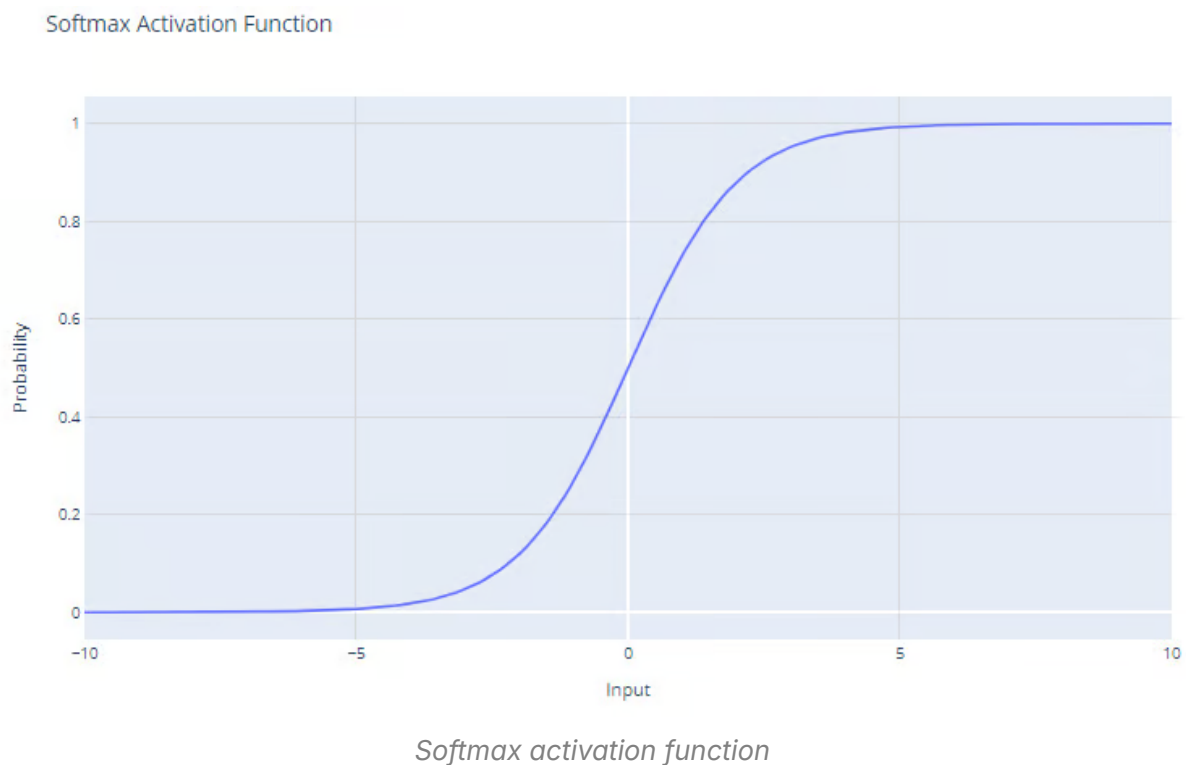
## Leaky ReLU:



It prevents dying ReLU problem. This variation of ReLU has a small positive slope in the negative area, so it does enable back-propagation, even for negative input values.

Leaky ReLU does not provide consistent predictions for negative input values. During the front propagation if the learning rate is set very high it will overshoot killing the neuron.

## Softmax activation



The softmax activation function, also known as the normalized exponential function, is particularly useful within the context of multi-class classification problems. This function operates on a vector, often referred to as the logits, which represents the raw predictions or scores for each class computed by the previous layers of a neural network.

For input vector  $x$  with elements  $x_1, x_2, \dots, x_C$ , the softmax function is defined as:

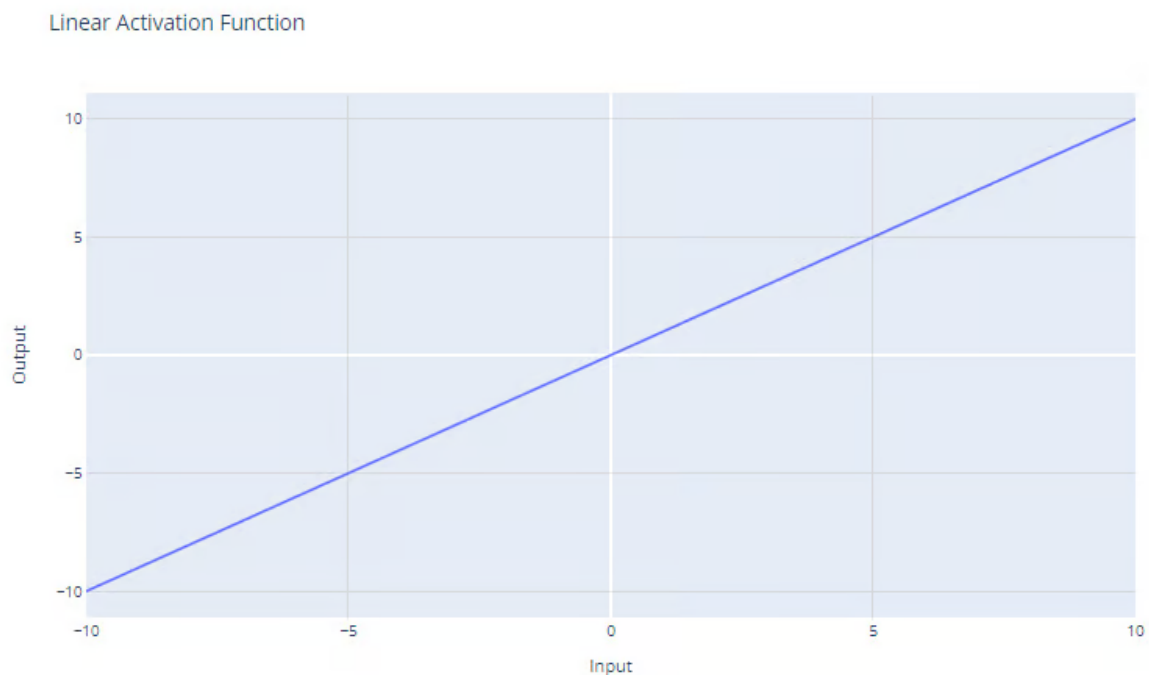
$$f(x_i) = e^{x_i} / \sum_j e^{x_j}$$

$$\text{softmax}(z_j) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}} \text{ for } j = 1, \dots, K$$

The output of the softmax function is a probability distribution that sums up to one. Each element of the output represents the probability that the input belongs to a particular class.

Softmax is typically used in the output layer of a neural network when the task involves classifying an input into one of several (more than two) possible categories (multi-class classification).

## Linear activation



*Linear activation function.*

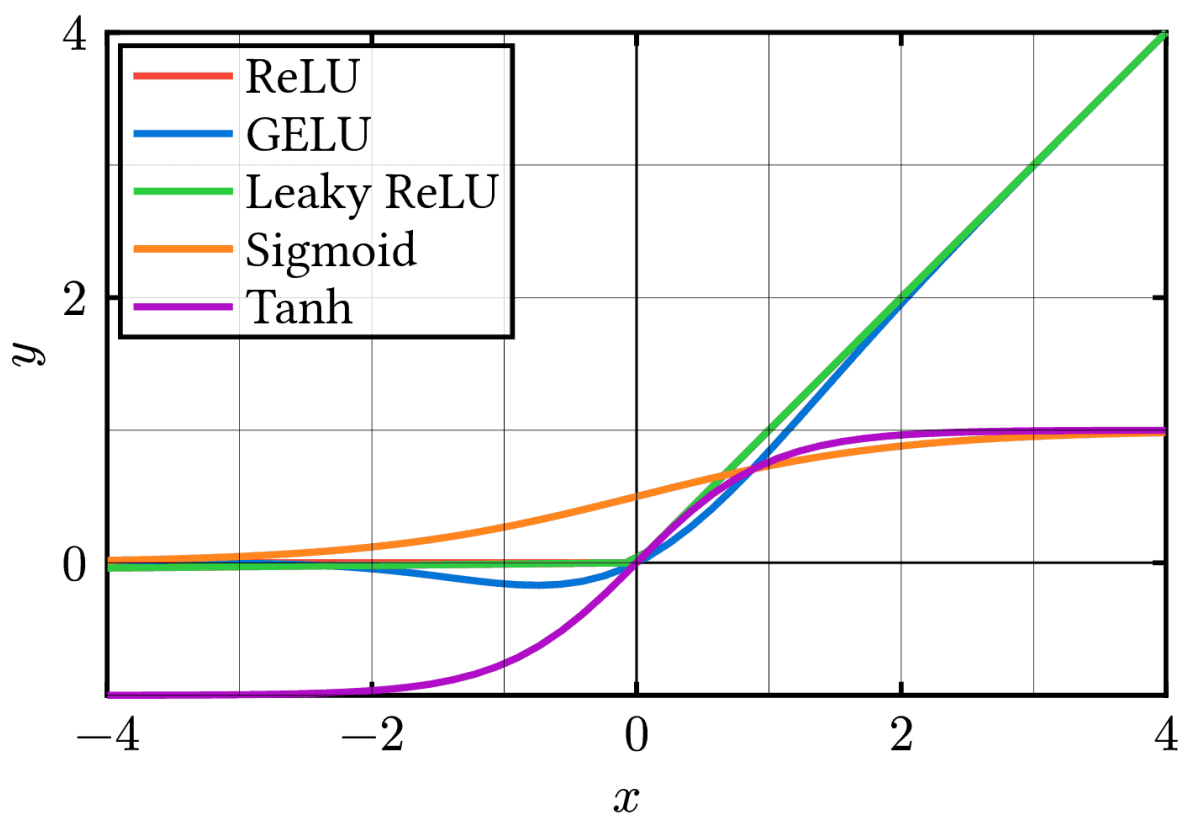
The linear activation function is the simplest activation function, defined as:

$$f(x) = x$$

It simply returns the input  $x$  as the output. Graphically, it looks like a straight line with a slope of 1.

For regression problems where we want to predict a numerical value, using a linear activation function in the output layer ensures the neural network outputs a numerical value. The linear activation function does not squash or transform the output, so the actual predicted value is returned.

It does not provide any non-linearity. The whole point of hidden layers is to learn non-linear combinations of the input features. Using a linear activation throughout would restrict the model to just learning linear transformations of the input.



units → no of input features

1. The activation function of all input and hidden layer neurons would be relu or Leaky Relu function. SIGMOID IS NEVER USED IN HIDDEN LAYERS.
2. For binary classification (response is 1/0 or Y/N), the number of output neurons is "ONE" and the activation function used in it is SIGMOID function.
3. For multi class classification (eg type of course or profession opted), the number of neurons in the output layer would be equal to the number of options and the activation function used in each neuron is SOFTMAX. SoftMax function can generate probabilities for multiple classes.
4. If the response is continuous data, the number of output neurons is ONE and the activation function used is LINEAR ACTIVATION.